

Продуктовый план GameAIEditor

От «работающего пайплайна» до инструмента, которым можно пользоваться каждый день не задумываясь. План разбит на 6 направлений — не строго последовательных, но именно в таком порядке они дают наибольший эффект относительно затрат.

1 игровой профиль (Arma Reforger)	16 пакетов в src/game_ai_editor	5 / 14 типов событий реально детектится	72 теста, зелёные
--------------------------------------	------------------------------------	--	----------------------

Уже в фундаменте

Не переделываем, опираемся на это.

- * Vision — опциональный слой, а не зависимость. vision.enabled: false по умолчанию в профиле игры; без него пайплайн живёт на motion + audio + транскрипции.
- * Провайдеры ИИ заменяемы — ollama / lm_studio / openrouter / custom за одним интерфейсом VisionProvider, без переписывания оркестратора.
- * Resumable sessions и batch — стадии кэшируются на диск, обработка продолжается после сбоя одного видео.
- * Windows-инсталлятор без прав администратора уже собирается и публикуется в релизах.
- * CI держит lint / типы / coverage — есть на чём основывать дальнейшие изменения, не боясь тихой деградации.

Дорожная карта

6 направлений

01 Качество детекции

ЗАПЛАНИРОВАНО усилие: M

Сейчас events/detector.py способен физически выдать только 5 типов события из 14, описанных в профиле игры (headshot, multi_kill, squad_coordination и другие никогда не появляются). Это главный разрыв между «пайплайн работает» и «монтаж выглядит осмысленно».

- Классификация типа события по ключевым словам транскрипта (рация/чат уже режется на сегменты — используется только как булев флаг эмоции, не как источник типа события).
- Аудио-сигнатуры для explosion / vehicle_destruction без анализа спектра, а не только через порог громкости.
- Пересчитать scoring_weights в config/games/arma_reforger.json — narrative_value и confidence сейчас плоские заглушки, рассчитанные на обогащение от Vision, которого по умолчанию нет.

02 Производительность

В ПРОЦЕССЕ

усилие: S

10 минут видео не должны анализироваться 20 минут. Первая половина уже сделана в этом рабочем цикле; вторая устраняет оставшееся дублирование работы.

* ~~motion.py переписан с покадрового OpenCV декода на FFmpeg pipe с downscale — было ~18000 честных декодирований кадра, стало один проход.~~

* ~~audio / motion / transcription стадии запускаются параллельно (ThreadPoolExecutor), а не одна за другой.~~

- Убрать дублирование: prefilter и motion сейчас читают одно и то же видео двумя отдельными проходами FFmpeg — можно скормить motion уже извлечённые prefilter-кадры.
- Выбор размера модели faster-whisper (tiny/base/small) — вынести в настройки десктопа, а не хардкодить «base».
- Зафиксировать в репозитории короткий реальный игровой клип как эталон для бенчмарка — иначе регресс скорости узнаётся от пользователя, а не от CI.

03 Качество монтажа и вывода

ЗАПЛАНИРОВАНО

усилие: L

Это то, что отличает «инструмент, который находит моменты» от «продукт, который отдаёт готовый ролик». Сейчас на выходе один формат MP4 без пресетов под платформы.

- Пресеты экспорта под соотношения сторон — 16:9 (YouTube), 9:16 (Shorts/Reels/TikTok), 1:1.
- Фоновая музыкальная дорожка с ducking под игровой звук — audio/ уже умеет нормализацию и ducking для голоса/эффектов, для музыки — нет.
- Управление стилем субтитров (шрифт, позиция, длительность показа), не только сама генерация.
- Слот под intro/outro и водяной знак — актуально для тех, кто публикует под своим каналом.

04 Дистрибуция и первый запуск

ЗАПЛАНИРОВАНО

усилие: M

Установщик уже есть, но между «скачал .exe» и «первый готовый ролик» ещё есть трение, которое отсекает часть пользователей до того, как они увидят ценность продукта.

- Подписанный инсталлятор + канал автообновлений — сейчас новая версия означает ручной поиск релиза на GitHub.
- Забандлить FFmpeg прямо в дистрибутив — это единственная жёсткая внешняя зависимость (в отличие от Vision/Ollama, которые уже опциональны), и самая частая причина падения на первом запуске.
- Опциональная (opt-in) телеметрия ошибок — без неё узнать о реальных сбоях можно только если пользователь сам напишет.

05 Расширение на другие игры

ЗАПЛАНИРОВАНО

усилие: M

ARCHITECTURE.md заявляет «архитектура достаточно общая для новых игр» — но это утверждение никогда не проверялось: в репозитории всего один профиль. Пока не появится второй, это гипотеза, а не факт.

- Задokumentировать контракт `config/games/*.json` как руководство по добавлению профиля — сейчас формат понятен только по чтению кода.
- Собрать второй профиль для игры с другой «грамматикой действия» (например, гонки или королевская битва) — это вскроет скрытые допущения, специфичные для Arma, если они есть.

06 Доверие и обратная связь

ЗАПЛАНИРОВАНО

усилие: S

Соло-разработка означает, что единственный источник сигнала о реальных проблемах — собственное использование продукта. Жалоба «20 минут на 10-минутное видео», с которой начался этот рабочий цикл, — пример такого сигнала. Стоит формализовать канал для него.

- Простой changelog по релизам — версии уже тегируются на GitHub, не хватает читаемой сводки «что изменилось».
- Журнал трения при использовании — короткие заметки по каждому реальному прогону на своих видео, не только синтетические тесты.

Решения нужны от тебя

Дальше — не то, что можно молча закодить: развилки, которые меняют форму продукта.

Позиционирование: локально-приватный инструмент или гибрид с облаком

README сейчас прямо заявляет «не коммерческий видеоредактор», local-first. Облачный provider по умолчанию снял бы трение установки Ollama, но противоречит текущему УТП «ничего не уходит наружу». Рекомендация: оставить local-first основным — это уже свершившийся факт архитектуры, а не гипотеза, и облачный тир добавлять только при реальном сигнале спроса, а не заранее.

Какая игра — вторая

Фаза 05 нужна независимо от ответа. Рекомендация: выбирать не по критерию «разная грамматика действия для проверки архитектуры», а игру, в которую сам реально играешь и будешь генерировать монтажи каждую неделю — догфудинг даст более честную обратную связь, чем синтетический выбор.